

ESPECIFICACIONES DE EVALUACION TECNICA

JULIO 2025

ÁREA RESPONSABLE:

DEPARTAMENTO DE TECNOLOGÍA DE LA INFORMACIÓN
SISTEMAS WEB Y REGULATORIO

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 2 de 8

Tabla de contenido

1.	Objetivo	3
2.	Alcance.....	3
3.	Desarrollo de aplicación web de gestión de pagos de empleados.....	3
4.	Preguntas de conceptualización	7
5.	Forma de entrega	8

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 3 de 8

1. Objetivo

Evaluar candidatos Full Stack con enfoque en C# .NET Core, React y bases de datos (SQL/Oracle), mediante pruebas diseñadas para validar su dominio técnico, buenas prácticas de desarrollo y experiencia en arquitecturas modernas.

2. Alcance

Esta evaluación tiene como objetivo validar las competencias técnicas de candidatos al rol de **desarrollador Full Stack**, con énfasis en el manejo de **C# .NET Core, React y bases de datos relacionales** como **SQL Server u Oracle**.

El alcance de la prueba abarca:

- Desarrollo de APIs RESTful utilizando .NET Core.
- Implementación de interfaces dinámicas con React.
- Escritura y optimización de consultas SQL (incluyendo modelado relacional).
- Aplicación de buenas prácticas de desarrollo (estructura limpia, control de versiones, patrones comunes).
- Conocimiento práctico de arquitecturas modernas, incluyendo separación de capas, integración de servicios y principios de escalabilidad.
- implementación de seguridad básica de utilizando JWT.

La evaluación está diseñada para medir tanto el **nivel técnico** como la **capacidad de estructurar soluciones eficientes y mantenibles**, considerando escenarios comunes en entornos de desarrollo empresarial.

3. Desarrollo de aplicación web de gestión de pagos de empleados

Objetivo: Evaluar conocimientos prácticos en arquitectura, patrones de diseño, integración frontend-backend, buenas prácticas y estructura de código.

Enunciado:

Desarrollar una aplicación que permita a la compañía calcular los pagos semanales de sus empleados, capturar y almacenar datos de los empleados, generar reportes, y los permisos básicos (admin/usuario).

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 4 de 8

La solución debe permitir:

- Gestionar los empleados
- Filtros por nombre, departamento y estado.
- Gestión de usuarios con roles (autenticación JWT).
- Gestión de cálculo de pago según el tipo de empleado (Empleado asalariado, por hora, por comisión y empleado por comisión).

Requisitos funcionales:

- **Gestión de empleados**
 - El sistema debe permitir la captura de los datos de los empleados según su tipo:
 - **Empleado Asalariado:**
 - Captura: primerNombre, apellidoPaterno, numeroSeguroSocial, salarioSemanal.
 - **Empleado por Horas:**
 - Captura: apellidoPaterno, numeroSeguroSocial, sueldoPorHora, horasTrabajadas.
 - **Empleado por Comisión:**
 - Captura: primerNombre, apellidoPaterno, numeroSeguroSocial, ventasBrutas, tarifaComision.
 - **Empleado Asalariado por Comisión:**
 - Captura: primerNombre, apellidoPaterno, numeroSeguroSocial, ventasBrutas, tarifaComision, salarioBase.

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 5 de 8

- **Cálculo de Pago**

- El sistema debe calcular automáticamente el pago semanal según el tipo de empleado:

1. **Empleado Asalariado**

- **Pago semanal** = salarioSemanal.

2. **Empleado por horas:**

- Si **horasTrabajadas** ≤ 40 , entonces el pago = sueldoPorHora $\times$ horasTrabajadas.
- Si **horasTrabajadas** > 40 , entonces el pago = (sueldoPorHora $\times 40$) + (sueldoPorHora $\times 1.5 \times$ (horasTrabajadas - 40)).

3. **Empleado por Comisión**

- Pago semanal = ventasBrutas $\times$ tarifaComision

4. **Empleado Asalariado por Comisión**

- Pago semanal = (ventasBrutas $\times$ tarifaComision) + salarioBase + (salarioBase $\times 0.10$).

- El sistema debe permitir actualizar la información de los empleados para recalcular el pago si es necesario.

- **Generación de los reportes**

- El sistema debe generar un reporte semanal con el pago de cada empleado, detallando los cálculos según el tipo de contrato.

- **Requisitos no funcionales:**

- **Usabilidad**

- La interfaz debe ser intuitiva y fácil de navegar para usuarios no técnicos.

- **Escalabilidad**

- La arquitectura del sistema debe ser escalable, permitiendo la inclusión de nuevos tipos de empleados y cálculos adicionales sin modificar el código existente.

- **Mantenibilidad**

- El sistema debe estar diseñado de manera modular para facilitar la adición o modificación de funcionalidades sin afectar otros módulos, como también fácil de mantener y adaptable a cambios.

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 6 de 8

- **Rendimiento**
 - El sistema debe procesar los cálculos de pago para hasta 1,000 empleados en menos de 2 segundos.
- **Testiabilidad**
 - Garantizar la capacidad de la aplicación para ser probado de manera controlada y reproducible.

Requisitos técnicos:

- **Backend:** .NET 8, C#, ASP.CORE APIs, Entity Framework Core.
- **Frontend:** React + TypeScript
- **Maquetación:** utilizar como referencia la imagen llamada Maqueta.jpg
- **Base de datos:** SQL Server u Oracle (a elección).
- **Arquitectura:** fácil de mantener a través del tiempo.

Notas de requisitos:

- Color azul: rgba(13, 48, 72, .9)
- Color gris: rgba(237, 240, 247)
- Icono: adjunto en svg
- El logo puede ser tomado desde nuestro portal web en el footer

Extras:

- Validaciones de datos.
- Pruebas unitarias (mínimo 2–3).
- README con instrucciones claras de ejecución.

Criterios de evaluación:

- Estructura del código y buenas prácticas (OOD y SOLID)
- Claridad de código y lógica implementada
- Uso de arquitectura de arquitectura limpia
- Conexión frontend-backend limpia y funcional
- Buen diseño UI/UX básico
- Seguridad mínima (auth, validaciones)
- Uso eficiente de base de datos (queries, relaciones)
- La aplicación debe loggear todo lo que pase.
- Claridad en el README y pruebas incluidas
- Uso de buenas prácticas en React y .NET

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 7 de 8

4. Preguntas de conceptualización

Explica cómo aplicarías Clean Architecture en un proyecto .NET.

¿Cómo garantizarías la seguridad en una API REST que maneja datos sensibles?

¿Cuándo usarías microservicios y cuándo monolito?

¿Qué diferencia hay entre Entity Framework y Dapper y cuándo usarías cada uno?

¿Cómo has trabajado en proyectos con equipos ágiles y qué herramientas has usado?

¿Cómo garantizas que el código de la aplicación puede ser probado?

¿Cómo evitarías la saturación de un API?

¿Cómo deben de comunicarse los componentes en una arquitectura basada en componentes?

Criterios de evaluación:

1. Claridad técnica y profundidad en respuestas escritas
2. Conexión y diseño del flujo frontend-backend
3. Nivel de reflexión arquitectónica y madurez técnica

	Documentación prueba técnica	CÓDIGO:
		# VERSIÓN: 2
		PÁGINA: 8 de 8

5. Forma de entrega

Aplicación:

- Subir el código a un repositorio Git público (GitHub, GitLab o Azure DevOps).
- Incluir un archivo README con instrucciones claras para ejecutar el proyecto.
- El backend debe funcionar localmente con .NET 7/8, y el frontend con React.
- Incluir scripts de base de datos o archivo .bak si aplica.
- Si no puedes usar Git, entrega el proyecto comprimido en un .zip con la misma estructura y documentación.

Base de Datos:

- Entregar un script .sql que permita crear las tablas y poblarlas con datos de ejemplo.
- Alternativamente, puedes incluir un archivo .bak (SQL Server) para restaurar la base.
- Si usas Entity Framework Core, incluir las migraciones o el código del DbContext con instrucciones para aplicar dotnet ef database update.